Students:
Nguyen Huu Truong Giang [u6110634@udg.edu]

Lab 3: Task-Priority kinematic control (1A)

1 Introduction

In controlling a robot arm, the robot sometimes possesses more degrees of freedom than is necessary to perform a task. Therefore, the robot can achieve more dexterous motions, and it may still be possible to complete the task even if one joint cannot move. Moreover, redundancy can be used to realize additional tasks through the Jacobian null-space projector.

In this lab, we introduce the task-priority algorithm, which uses null-space motions to perform secondary tasks without affecting the primary task in a 2D planar robotic arm with three revolute joints. We conduct experiments with two tasks: the first task is to move the end effector to a desired position, and the secondary task is to keep a specific joint of the robot at a fixed position.

Section 2 presents the formulation of the task-priority algorithm used to calculate the kinematic control of the robot arm to complete the predefined tasks. Section 3 illustrates the results under different scenarios and discusses the key observations from the experiments. Section 4 answers the discussion questions. Finally, Section 5 concludes the lab.

2 Methodology

The implementation was carried out in Python using the `NumPy` library for numerical computations and `Matplotlib` for visualizing the robot motion and results. The null-space motion equation, recursive task-priority formulation, and velocity scaling are described below.

2.1 Null-space motion

For a redundant system, the robot can complete a task with different configurations. The redundant equation system is presented below together with the null-space projector:

$$\zeta = J^{\dagger}(q)\dot{x}_E + \left(I - J^{\dagger}(q)J(q)\right) y$$

where:

- $J^{\dagger}$ is the pseudoinverse of the robot Jacobian matrix,
- $\dot{x}_E$ is the Cartesian end-effector twist,
- $\left(I - J^{\dagger}(q)J(q)\right)$ is the null-space projector,
- y is an arbitrary velocity vector.

Appendix A.1 shows the Python implementation of the null-space projector and the redundant equation system.

2.2 Recursive task-priority formulation

We can also take advantage of system redundancy to perform secondary tasks without affecting the primary task by using null-space motion. The recursive task-priority formulation allows the combination of multiple tasks with different priorities in a priority list. The algorithm is described below:

Algorithm 1 Task-Priority Algorithm

Input: List of tasks $\{J_i(q), \dot{x}_i(q)\},\ i = 1 \dots k$
Output: Quasi-velocities $\zeta_k \in \mathbb{R}^n$
Initialise: $\zeta_0 = 0^n, \quad P_0 = I^{n\times n}$
for $i = 1$ to k **do**
    $\bar{J}_i(q) = J_i(q)P_{i-1}$
    $\zeta_i = \zeta_{i-1} + \bar{J}_i^{\dagger}(q)\left(\dot{x}_i(q) - J_i(q)\zeta_{i-1}\right)$
    $P_i = P_{i-1} - \bar{J}_i^{\dagger}(q)\bar{J}_i(q)$
end for
return ζ_k

The Python implementation of this algorithm is shown in Appendix A.2.

2.3 Velocity scaling/limiting

The quasi-velocities produced by the task-priority algorithm may not always be feasible or safe for the physical robotic platform. Velocity limits are commonly defined for each degree of freedom (DOF) and are closely related to the performance of the actuators. Therefore, the velocities are scaled to ensure that the velocity of each joint does not exceed its predefined limit. The velocity scaling method used to achieve this is described below:

Algorithm 2 Velocity Scaling

$s = \max_{i\in 1\dots n}\left(\frac{|\zeta_i|}{\dot{\zeta}_{i,\max}}\right)$
if $s > 1$ **then**
    return $\frac{\zeta}{s}$
else
    return ζ
end if

The Python implementation of the velocity scaling procedure is provided in Appendix A.3.

3 Results

Two main experiments are conducted in this study. In the first experiment, we implement the redundant system equation and apply an arbitrary velocity while the robot still achieves the desired end-effector position in order to observe how the robot moves.

In the second experiment, two tasks are defined: moving the end effector to the desired position and keeping joint 1 at a desired position. We apply the recursive task-priority formulation with two different priority configurations. In the first case, achieving the desired end-effector position has the highest priority, while in the second case, maintaining the position of joint 1 has the highest priority. The errors of the end-effector position and the joint 1 position are then plotted to analyze the behavior of the robot arm.

In this laboratory experiment, a 3-DOF planar robot with three revolute joints is used. The robot model, including its coordinate frames and Denavit–Hartenberg (DH) parameters, is illustrated in Fig. 1.

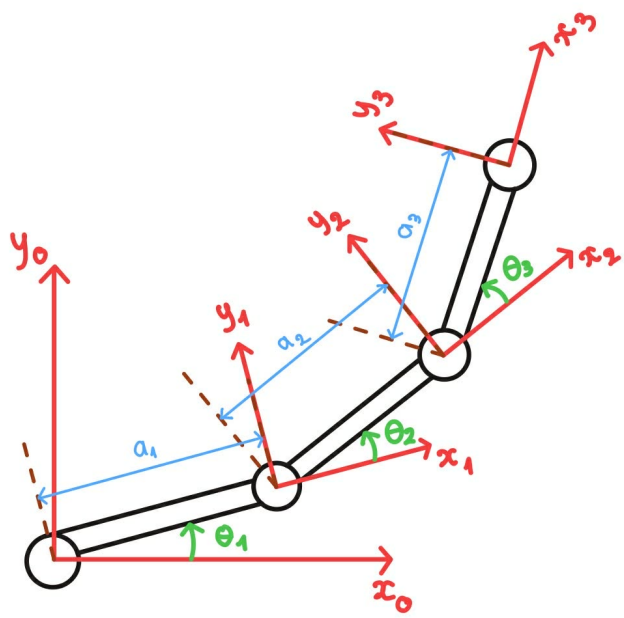

Figure 1: Visualization of the 3-DOF planar robot model

The robot configuration is initialized according to the DH parameters listed in Table 1.

DOF	θ_i	d_i	a_i	α_i	Home
1	q_1	0	$a_1 = 0.5$	0	0.2
2	q_2	0	$a_2 = 0.5$	0	0.5
3	q_3	0	$a_3 = 0.5$	0	0.2

Table 1: Denavit–Hartenberg parameters of the 3-DOF robot arm

3.1 Null-Space Projector

To conduct the first experiment, the arbitrary velocity vector equation, the gain parameters of the robot arm, the total simulation time, and the sampling interval are initialized. The parameters used in the simulation are summarized in Table 2.

Arbitrary velocity	Gain parameter	Sampling time	Total simulation time
$[\sin t, \sin 2t, \sin 3t]$	$[1.0, 1.0]$	$1/60$	60

Table 2: Parameters for the null-space projector experiment

After setting these parameters, the robot arm is simulated while maintaining the end-effector at the desired position, whereas the other joints continue moving. The robot motion, end-effector trajectory, and joint positions over time are recorded. The visualization of the robot motion and the end-effector path is shown in Fig. 2, while the evolution of joint positions over time is shown in Fig. 3.

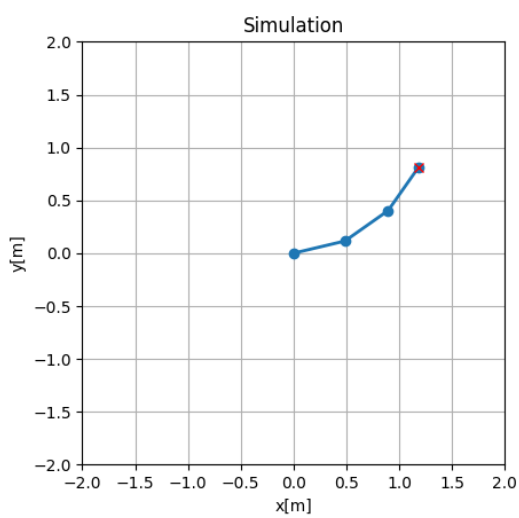

Figure 2: Simulation of the planar robot and the end-effector trajectory

As shown in Fig. 3, the joint positions exhibit periodic behavior because the arbitrary velocity vector is defined using sinusoidal functions. Since each joint uses a sinusoidal function with a different frequency, the resulting motion of each joint has a different period. However,

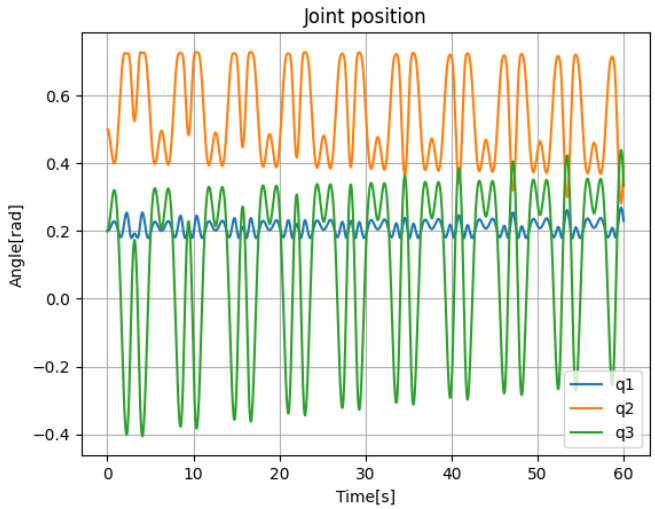

Figure 3: Joint positions of the robot during the simulation

the end-effector remains at the desired position because the arbitrary velocity acts within the null space of the primary task, while the primary task maintains the end-effector at the desired velocity.

The full Python code used to run the experiment is provided in Appendix A.4.

3.2 Recursive Task-Priority

In this experiment, two tasks are defined: moving the end effector to the desired position and maintaining joint 1 at a desired position. The experiment is divided into two cases. In Case A, the end-effector task is assigned higher priority, whereas in Case B, the joint 1 position task is assigned higher priority. This allows us to observe how the robot behaves under different priority configurations.

Before conducting the experiment, several parameters are defined. These parameters are listed in Table 3.

Desired joint 1 position	0.0
Gain for end-effector task	$[1.0, 1.0]$
Gain for joint position task	1.0
Maximum joint velocities	$[0.9, 1.0, 0.9]$
Sampling time	$1/60$
Total simulation time	10
Damping factor	0.2

Table 3: Parameters for the task-priority experiment

3.2.1 End-effector position as the first priority

In this experiment, the end-effector task is given higher priority than the joint position task. After conducting the experiment, we obtain the robot motion and end-effector trajectory (Fig. 4), as well as the plots of the errors between the end-effector position and the desired point and the joint 1 position error (Fig. 5).

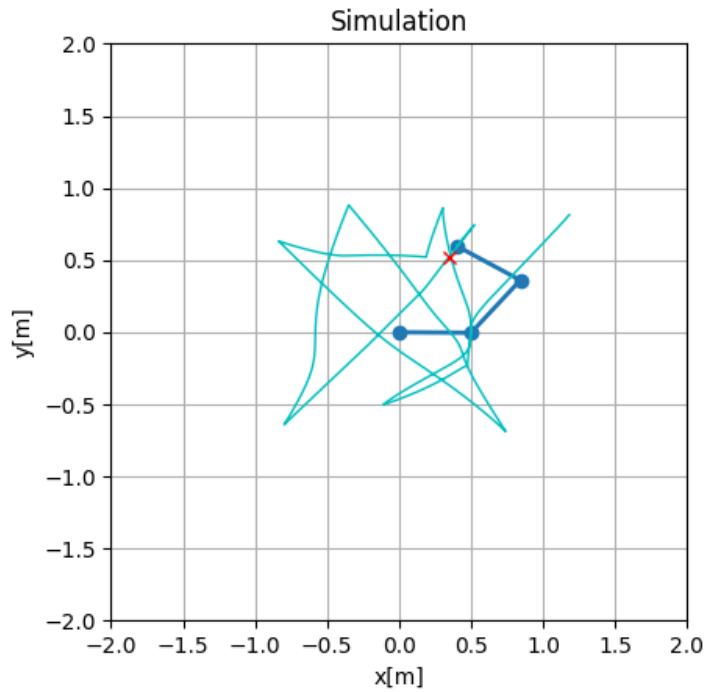

Figure 4: Simulation of the manipulator, including end effector in case high priority of end effector

As shown in Fig. 5, the end-effector position error decreases to zero every ten seconds because a new desired point is generated every ten seconds, and the robot attempts to reach it. However, the joint 1 position error does not always converge to zero because achieving the desired end-effector position may require joint 1 to move away from its desired position. Since the end-effector task has higher priority, the joint position task cannot always be satisfied.

The full Python code implementation for this case is shown in Appendix A.5.

3.2.2 Joint position as the first priority

In contrast to the previous case, this experiment assigns higher priority to the joint 1 position task. As a result, the behavior of the robot differs from the case where the end-effector position task has higher priority. The simulation of the manipulator and the plots of the end-effector error and joint 1 position error are shown in Fig. 6 and Fig. 7, respectively.

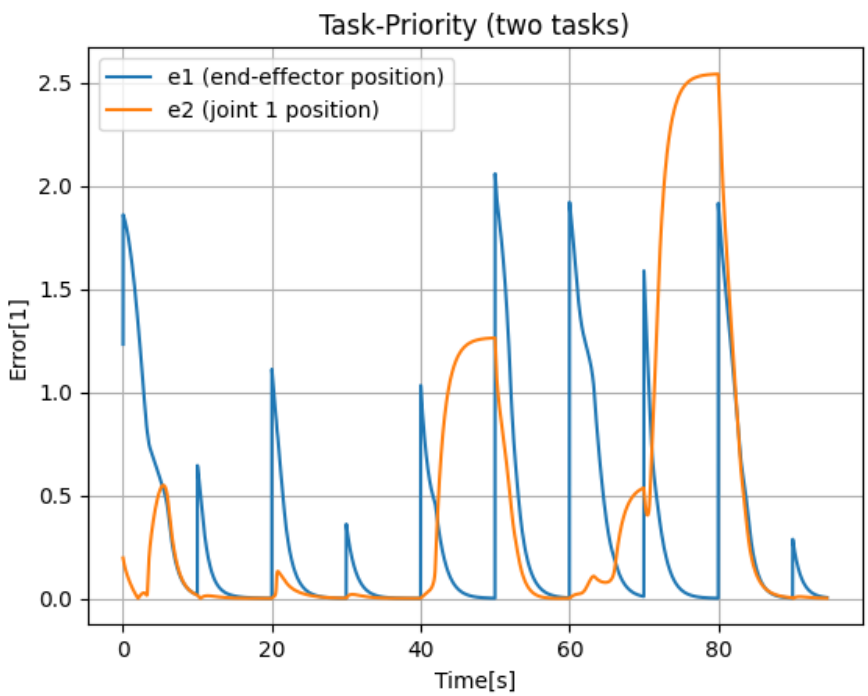

Figure 5: Evolution of the TP control errors in case high priority of end effector

From Fig. 7, we can observe that the robot prioritizes maintaining joint 1 at the desired position. Therefore, the joint 1 position error remains close to zero throughout the simulation. However, the end-effector may not always reach the desired point because the primary task constrains the motion of the robot, as shown in the blue line (end effector position error) and orange line (joint 1 position error) sometimes the blue line cannot converge to zero every ten seconds.

The full Python code implementation for this case is shown in Appendix A.6.

4 Discussion Questions

Question 1: What are the advantages and disadvantages of redundant robotic systems?

Answer: Redundant robotic system offer several advantages such as the robot has more degrees of freedom that are necessary to perform the task so we can add additional task for the robot through using null space and task-priority algorithm. Moreover, the robot can achieve more dexterous motions and if one its joint is fail the robot can still complete the task.

However, the redundant robotic systems also presents several disadvantages. One of those is the increased computational complexity since additional algorithms we have to add such as null space projector and task-priority algorithm. Another problem is that the redundant system can

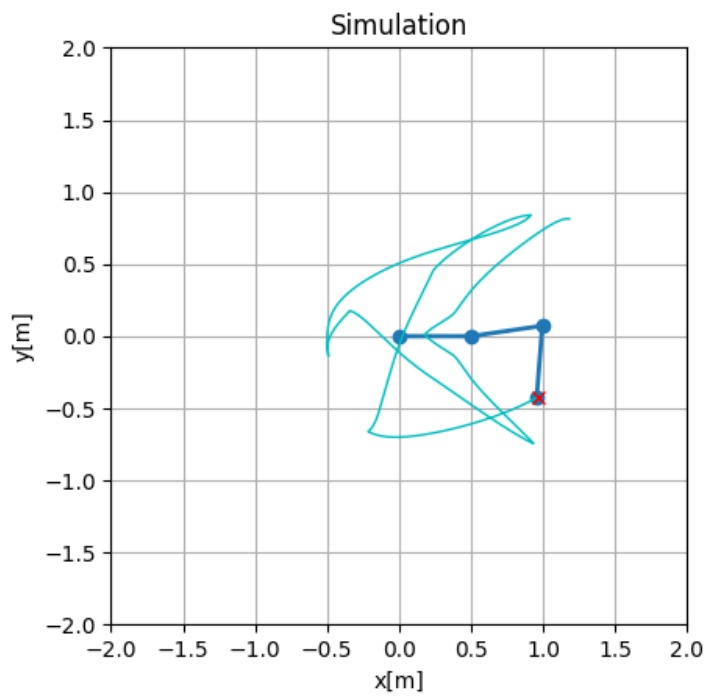

Figure 6: Simulation of the manipulator, including end effector in case high priority of joint position

introduce multiple solutions for the same task which may lead to instability and undesirable motion if the control strategy is not designed correctly. Finally, redundant robotic system requires more complex mechanical design and higher control effort compared to non-redundant robots.

Question 2: What is the meaning and practical use of a weighting matrix W, that can be introduced in the pseudoinverse/DLS implementation?

Answer: A weighting matrix W is used in the pseudoinverse or Damped Least Squares (DLS) implementation to assign different importance to the degrees of freedom (DOF) of the robotic system. By introducing this matrix, the controller can prefer or penalize the motion of certain joints when computing the solution of the inverse kinematics problem.

In practice, the weighting matrix allows the optimization of different criteria, such as reducing energy consumption, avoiding excessive joint motion, or improving convergence behavior. A higher weight assigned to a specific DOF means that the algorithm will penalize the motion of that joint, making it less preferred in the resulting solution.

One practical example occurs in mobile manipulation systems, the weighting matrix can be designed as a function of the distance between the end effector and the target position. When the robot is far from the target, the weights can be selected so that the motion of the mobile base is

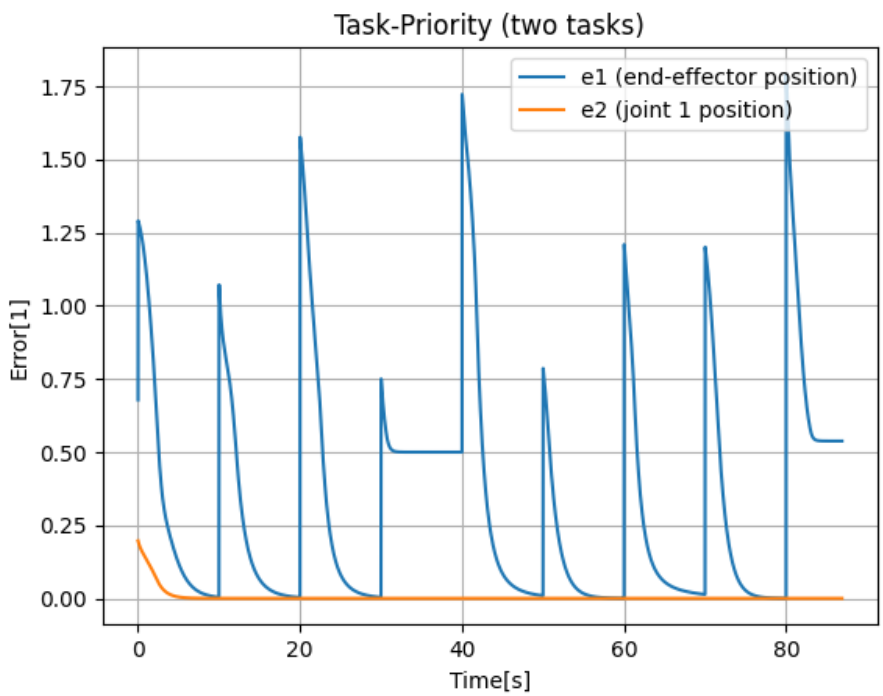

Figure 7: Evolution of the TP control errors in case high priority of joint position

preferred, allowing the robot to move closer to the goal. As the robot approaches the target, the weights can be adjusted to favor the manipulator joints, enabling the robotic arm to perform the fine positioning of the end effector at the desired point.

5 Conclusion

In conclusion, this laboratory exercise demonstrates the implementation of the null space projector and the task-priority algorithm. We utilized the redundancy of the robotic system to introduce additional tasks through the use of the null space projector and the task-priority formulation.

Through simulation and analysis of the end-effector position error and the joint position error, it can be observed that the robot behavior depends on the defined priority list. The robot can either accomplish both tasks or only satisfy the higher priority task when the tasks conflict with each other.

This algorithm is particularly important for redundant robotic systems, as it allows the robot to accomplish certain tasks even without using one or more joints, which may be unavailable due to failures or constraints.

A Appendix: Code Implementations

This appendix contains all Python code implementations used in the laboratory exercise. To improve clarity and readability of the main report, all figures presenting source code are moved to this appendix. These implementations include null space projector, recursive task-priority, the code for plotting figure and the complete code for each experiments.

A.1 Null space projector

```
# Update robot
T = kinematics(d, q.flatten(), a, alpha)
J = jacobian(T, revolute)

# Update control
sigma = T[-1][0:2, 3].reshape(2,1)   # Current position of the end-effector
err = sigma_d - sigma                # Error in position
Jbar = J[0:2][:]                     # Task Jacobian
P = (np.identity(3) - np.linalg.pinv(Jbar) @ Jbar)       # Null space projector
y = np.array([sin(t), sin(2*t), sin(3*t)]).reshape(3,1)  # Arbitrary joint velocity
dq = np.linalg.pinv(Jbar) @ (K @ err) + P @ y            # Control signal
q = q.reshape(3,1) + dt * dq                             # Simulation update
q = q.flatten()
```

A.2 Recursive task-priority

```
# Update robot
T = kinematics(d, q.flatten(), a, alpha)
J = jacobian(T, revolute)

# Calculate DLS
damping_factor = 0.2

# Update control
# TASK 1
sigma1 = T[-1][0:2, 3].reshape((2,1))             # Current position of the end-effector
err1 = sigma1_d - sigma1                          # Error in Cartesian position
J1 = J[0:2][:]                                    # Jacobian of the first task
DLS_matrix_J1 = DLS(J1, damping_factor)
P1 = (np.identity(3) - np.linalg.pinv(J1) @ J1)   # Null space projector

# TASK 2
sigma2 = q[0]                            # Current position of joint 1
err2 = sigma2_d - sigma2                 # Error in joint position
J2 = np.array([1, 0, 0]).reshape(1,3)    # Jacobian of the second task
```

```
J2bar = J2 @ P1                          # Augmented Jacobian
DLS_matrix_J2 = DLS(J2bar, damping_factor)

# Combining tasks
dq1 = DLS_matrix_J1 @ (K1 @ err1)                      # Velocity for the first task
dq12 = dq1 + DLS_matrix_J2 @ ((K2 @ err2) - J2 @ dq1)  # Velocity for both tasks
```

A.3 Velocity scaling/limiting

```
# Scaling factor to ensure joint velocity limits are not exceeded
s = np.max(np.abs(dq12) / dq_max)
if s > 1:
    dq12 = dq12 / s # Scale down the velocity to respect joint limits
else:
    dq12 = dq12 # No scaling needed
```

A.4 Full code for Experiment Null Space Projector

```
# Import necessary libraries
from lab2_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim
import math

# Robot definition (3 revolute joint planar manipulator)
d = np.zeros(3)                             # displacement along Z-axis
q = np.array([0.2, 0.5, 0.2])               # rotation around Z-axis (theta)
a = np.array([0.5, 0.5, 0.5])               # displacement along X-axis
alpha = np.zeros(3)                         # rotation around X-axis
revolute = np.array([True, True, True])     # flags specifying the type of joints
K = np.diag([1, 1]) # Control gain matrix

# Record the motion of three joints
q1_record = [q[0]]
q2_record = [q[1]]
q3_record = [q[2]]

# Setting desired position of end-effector to the current one
T = kinematics(d, q.flatten(), a, alpha) # flatten() needed if q defined as column vector
sigma_d = T[-1][0:2,3].reshape(2,1)

# Simulation params
dt = 1.0/60.0             # Time step
Tt = 10                   # Total simulation time
```

```
tt = np.arange(0, Tt, dt) # Simulation time vector

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
ax.grid()
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Simulation initialization
def init():
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    return line, path, point

# Simulation loop
def simulate(t):
    global q, a, d, alpha, revolute, sigma_d
    global PPx, PPy

    # Update robot
    T = kinematics(d, q.flatten(), a, alpha)
    J = jacobian(T, revolute)

    # Update control
    sigma = T[-1][0:2, 3].reshape(2,1)    # Current position of the end-effector
    err = sigma_d - sigma                                    # Error in position
    Jbar = J[0:2][:]                                         # Task Jacobian
    P = (np.identity(3) - np.linalg.pinv(Jbar) @ Jbar)       # Null space projector
    y = np.array([sin(t), sin(2*t), sin(3*t)]).reshape(3,1)  # Arbitrary joint velocity
    dq = np.linalg.pinv(Jbar) @ (K @ err) + P @ y            # Control signal
    q = q.reshape(3,1) + dt * dq                             # Simulation update
    q = q.flatten()
```

```
    # Record position of joints
    q1_record.append(q[0])
    q2_record.append(q[1])
    q3_record.append(q[2])

    # Update drawing
    PP = robotPoints2D(T)
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data([sigma_d[0]], [sigma_d[1]])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 60, dt),
                                interval=10, blit=True, init_func=init, repeat=False)
plt.show()

# Plot joint positions
t_rec = np.arange(len(q1_record)) * dt # Time vector for recorded joint angles
plt.plot(t_rec, q1_record, label='q1')
plt.plot(t_rec, q2_record, label='q2')
plt.plot(t_rec, q3_record, label='q3')
plt.legend()
plt.title('Joint position')
plt.xlabel('Time[s]')
plt.ylabel('Angle[rad]')
plt.grid()
plt.show()
```

A.5 Full code for Experiment Task-Priority with High Priority for End-Effector Position

```
# Import necessary libraries
from lab2_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim

# Robot definition (3 revolute joint planar manipulator)
d = np.zeros(3)                             # displacement along Z-axis
q = np.array([0.2, 0.5, 0.2])               # rotation around Z-axis (theta)
```

```
a = np.array([0.5, 0.5, 0.5])              # displacement along X-axis
alpha = np.zeros(3)                        # rotation around X-axis
revolute = np.array([True, True, True])    # flags specifying the type of joints
K1 = np.diag([1, 1])                       # Control gain matrix for the first task
K2 = np.diag([1])                          # Control gain matrix for the second task
dq_max = np.array([0.9, 1.0, 0.9])         # Maximum joint velocities

# Record error of ee and joint 1 position
err1_record = []
joint_1_record = []

# Desired values of task variables
sigma1_d = np.array([0.0, 1.0]).reshape(2,1) # Desired position of the end-effector
sigma2_d = np.array([[0.0]])                 # Desired position of joint 1

# Simulation params
dt = 1.0/60.0
Tt = 10 # Total simulation time
tt = np.arange(0, Tt, dt) # Simulation time vector

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
ax.grid()
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Simulation initialization
def init():
    global sigma1_d
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    sigma1_d = np.random.rand(2,1) * 2 - 1 # Random target position in the range [-1, 1]
    return line, path, point
```

```
# Simulation loop
def simulate(t):
    global q, a, d, alpha, revolute, sigma1_d, sigma2_d
    global PPx, PPy

    # Update robot
    T = kinematics(d, q.flatten(), a, alpha)
    J = jacobian(T, revolute)

    # Calculate DLS
    damping_factor = 0.2

    # Update control
    # TASK 1
    sigma1 = T[-1][0:2, 3].reshape((2,1))      # Current position of the end-effector
    err1 = sigma1_d - sigma1                   # Error in Cartesian position
    J1 = J[0:2][:]                             # Jacobian of the first task
    DLS_matrix_J1 = DLS(J1, damping_factor)
    P1 = (np.identity(3) - np.linalg.pinv(J1) @ J1) # Null space projector

    # TASK 2
    sigma2 = q[0]                           # Current position of joint 1
    err2 = sigma2_d - sigma2                # Error in joint position
    J2 = np.array([1, 0, 0]).reshape(1,3)   # Jacobian of the second task
    J2bar = J2 @ P1                         # Augmented Jacobian
    DLS_matrix_J2 = DLS(J2bar, damping_factor)

    # Combining tasks
    dq1 = DLS_matrix_J1 @ (K1 @ err1)                      # Velocity for the first task
    dq12 = dq1 + DLS_matrix_J2 @ ((K2 @ err2) - J2 @ dq1)  # Velocity for both tasks

    # Scaling factor to ensure joint velocity limits are not exceeded
    s = np.max(np.abs(dq12) / dq_max)
    if s > 1:
        dq12 = dq12 / s # Scale down the velocity to respect joint limits
    else:
        dq12 = dq12 # No scaling needed

    q = q.reshape(3,1) + dq12 * dt # Simulation update
    q = q.flatten()
```

```
    err1_record.append(np.linalg.norm(err1))
    joint_1_record.append(abs(q[0]))

    # Update drawing
    PP = robotPoints2D(T)
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(sigma1_d[0], sigma1_d[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init, repeat=True)
plt.show()

# Plot error of end-effector and joint 1 position
t_rec = np.arange(len(err1_record)) * dt # Time vector for recorded joint angles
plt.plot(t_rec, err1_record, label='e1 (end-effector position)')
plt.plot(t_rec, joint_1_record, label='e2 (joint 1 position)')
plt.legend()
plt.title('Task-Priority (two tasks)')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.grid()
plt.show()
```

A.6 Full code for Experiment Task-Priority with High Priority for Joint Position

```
# Import necessary libraries
from lab2_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim

# Robot definition (3 revolute joint planar manipulator)
d = np.zeros(3)                          # displacement along Z-axis
q = np.array([0.2, 0.5, 0.2])            # rotation around Z-axis (theta)
a = np.array([0.5, 0.5, 0.5])            # displacement along X-axis
alpha = np.zeros(3)                      # rotation around X-axis
revolute = np.array([True, True, True])  # flags specifying the type of joints
```

```
K1 = np.diag([1])                          # Control gain matrix for the first task
K2 = np.diag([1, 1])                       # Control gain matrix for the second task
dq_max = np.array([0.9, 1.0, 0.9])         # Maximum joint velocities

# Record error of ee and joint 1 position
err1_record = []
joint_1_record = []

# Desired values of task variables
sigma1_d = np.array([[0.0]])                 # Position of joint 1
sigma2_d = np.array([0.0, 1.0]).reshape(2,1) # Position of the end-effector

# Simulation params
dt = 1.0/60.0
Tt = 10 # Total simulation time
tt = np.arange(0, Tt, dt) # Simulation time vector

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
ax.grid()
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Simulation initialization
def init():
    global sigma2_d
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    sigma2_d = np.random.rand(2,1) * 2 - 1 # Random target position in the range [-1, 1]
    return line, path, point

# Simulation loop
def simulate(t):
```

```
global q, a, d, alpha, revolute, sigma1_d, sigma2_d
global PPx, PPy

# Update robot
T = kinematics(d, q.flatten(), a, alpha)
J = jacobian(T, revolute)

# Calculate DLS
damping_factor = 0.2

# Update control
# TASK 1
sigma1 = q[0]                              # Current joint 1 position
err1 = sigma1_d - sigma1                   # Error in joint position
J1 = np.array([1, 0, 0]).reshape(1,3)      # Jacobian of the joint position task
DLS_matrix_J1 = DLS(J1, damping_factor)
P1 = (np.identity(3) - np.linalg.pinv(J1) @ J1)  # Null space projector

# TASK 2
sigma2 = T[-1][0:2, 3].reshape((2,1))   # Current position of the end-effector
err2 = sigma2_d - sigma2                # Error in Cartesian position
J2 = J[0:2][:]                          # Jacobian of the second task
J2bar = J2 @ P1                         # Augmented Jacobian
DLS_matrix_J2 = DLS(J2bar, damping_factor)

# Combining tasks
dq1 = DLS_matrix_J1 @ (K1 @ err1)                      # Velocity for the first task
dq12 = dq1 + DLS_matrix_J2 @ ((K2 @ err2) - J2 @ dq1) # Velocity for both tasks

# Scaling factor to ensure joint velocity limits are not exceeded
s = np.max(np.abs(dq12) / dq_max)
if s > 1:
    dq12 = dq12 / s
else:
    dq12 = dq12

q = q.reshape(3,1) + dq12 * dt # Simulation update
q = q.flatten()

err1_record.append(np.linalg.norm(err2))
joint_1_record.append(abs(q[0]))
```

```
    # Update drawing
    PP = robotPoints2D(T)
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(sigma2_d[0], sigma2_d[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init, repeat=True)
plt.show()

# Plot error of end-effector and joint 1 position
t_rec = np.arange(len(err1_record)) * dt # Time vector for recorded joint angles
plt.plot(t_rec, err1_record, label='e1 (end-effector position)')
plt.plot(t_rec, joint_1_record, label='e2 (joint 1 position)')
plt.legend()
plt.title('Task-Priority (two tasks)')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.grid()
plt.show()
```